



Core: Audit Diff Check Doc

Comparison: `audit-main` (pre-audit) → `audit-main-fixes` (post-audit)

This document details every contract-level change made during the security audit. Organized by contract. Test-only changes are excluded.

UniversalCore.sol

1. Granular Role Separation
2. depositPRC20Token: Return Values from PRC20 Ignored + Missing Reentrancy Guard
3. depositPRC20TokenWithAutoSwap: Return Values from PRC20 Ignored
4. swapAndBurnGas: Inconsistent ERC-20 Handling + Return Values from PRC20 Ignored
5. getOutboundTxGasAndFees: gasLimitUsed Return + Stale Gas Data + Zero Base Gas Limit
6. getRescueFundsGasLimit: Stale Gas Data
7. setChainMeta: Zero Gas Price Allowed
8. updateGasTokenPRC20 (was setGasTokenPRC20): Gas Token Update Doesn't Invalidate Gas Price
9. updateDefaultFeeTier (was setDefaultFeeTier): Rejects Valid 1bps Fee Tier
10. updateUniswapV3Addresses (was setUniswapV3Addresses): uniswapV3Quoter Stored but Never Used
11. unpause: Role Separation
12. rescueNativePC (NEW): PC Can Become Permanently Stuck
13. updateMaxStalenessByChain (NEW): Stale Gas Price Data Accepted Without Freshness Validation
14. _validateGasDataFreshness (NEW): Stale Gas Price Data Accepted Without Freshness Validation
15. _autoSwap: Inconsistent ERC-20 Handling + Return Values Ignored
16. Removed: setSupportedToken, setSlippageTolerance, setPauserRole, onlyAdmin modifier
17. All function renames (set* → update*)
18. New events added: Missing Events for Critical Admin Changes

CEA.sol

19. Storage layout change: Removed direct VAULT+UG addresses
20. initializeCEA: Removed direct VAULT+UG addresses
21. onlyVault modifier: Live lookup from factory
22. sendUniversalTxToUEA: Single-Call Dual Semantics + ERC-20 Approval Reset + SafeERC20 Unused
23. _handleMulticall: Multicall Discards Underlying Revert Reasons
24. _handleSingleCall: Single-Call Dual Semantics + Revert Reasons
25. _handleMigration: CEA Migration Does Not Enforce Self-Targeting
26. executeUniversalTx param rename: txId → subTxId

CEAFactory.sol

27. Granular Role Separation
28. deployCEA: CEA init params simplified

29. Function renames and role reassignments

30. Removed: setPauserRole, PauserRoleGranted event

UEAFactory.sol

31. Granular Role Separation

32. getOriginForUEA: Hardcoded Push Chain ID

33. registerUEA: Can Overwrite Existing VM Implementation

34. updateUEAImplementation (NEW): Explicit update path for VM implementations

35. updatePushChainId (NEW): Configurable Push Chain ID

36. Function renames and role reassignments

37. Removed: setPauserRole, PauserRoleGranted event

UEA_EVM.sol

38. domainSeparator: Domain Separator Missing block.chainid

39. executeUniversalTx: payload.nonce Ignored During Signature Hashing

UEA_SVM.sol

40. domainSeparator: SVM Domain Separator Encodes chainId Incorrectly + Missing block.chainid

41. executeUniversalTx: payload.nonce Ignored During Signature Hashing

UEAProxy.sol

42. initializeUEA: Proxy Initialization Validation Inconsistent

PRC20.sol

43. transferFrom: Balance Transfer Before Allowance Validation

44. deposit: PRC20 Direct Mint Bypasses UniversalCore Pause

45. setName / setSymbol: Missing Events

WPC.sol

46. totalSupply: totalSupply Inflated by Force-Sent Native PC

47. deposit / withdraw: Supply tracking + withdraw Uses .transfer (2300 Gas)

48. transferFrom: Empty Revert Messages

Errors.sol (libraries)

49. New errors added

Utils.sol (StringUtils)

50. stringToExactUInt256: Custom Errors for Gas Efficiency

TESTNET: UniversalCoreV0.sol (testnetV0 — storage-compatible upgrade)

51. Deprecated state variables

52. setChainMeta: signature change (removed `observedAt` param)

53. All function renames (set* → update*)

54. Removed: setSupportedToken, setSlippageTolerance

55. Fee tier constants added

TESTNET: UEAFactoryV0.sol (testnetV0)

56. getOriginForUEA: configurable pushChainId

57. New: updatePushChainId, pushChainId state variable

58. Function renames (set* → update*)

59. PauserRoleGranted event moved to contract body

Review Tasks

UniversalCore.sol

Updated Testnet Version of UVCORE - <https://github.com/pushchain/push-chain-core-contracts/blob/audit-main-fixes/src/testnetV0/UniversalCoreV0.sol>

1. Granular Role Separation

Refer to the full Access Control design document:  [Push Smart Contracts - AccessControl](#)

2. depositPRC20Token: Return Values from PRC20 Ignored + Missing Reentrancy Guard

Before:

```
function depositPRC20Token(address prc20, uint256 amount, address recipient) external onlyUEModule whenNotPaused {
    _validateParams(prc20, amount, recipient);
    IPRC20(prc20).deposit(recipient, amount);
}
```

After:

```
function depositPRC20Token(address prc20, uint256 amount, address recipient)
    external
    onlyUEModule
    whenNotPaused
    nonReentrant
{
    _validateParams(prc20, amount, recipient);
    if (!IPRC20(prc20).deposit(recipient, amount)) revert UniversalCoreErrors.PRC20operationFailed();
}
```

Added `nonReentrant` modifier. PRC20 return value now checked — reverts with `PRC20operationFailed()` on failure.

3. depositPRC20TokenWithAutoSwap: Return Values from PRC20 Ignored

Before:

```
IPRC20(gasToken).deposit(recipient, amount);
```

After:

```
if (!IPRC20(gasToken).deposit(recipient, amount)) revert UniversalCoreErrors.PRC20operationFailed();
```

PRC20 return value check added in the non-swap branch.

4. swapAndBurnGas: Inconsistent ERC-20 Handling + Return Values from PRC20 Ignored

Before:

```
IERC20(WPC).approve(uniswapV3SwapRouter, msg.value);  
// ... swap ...  
IERC20(WPC).approve(uniswapV3SwapRouter, 0);  
IPRC20(gasToken).burn(gasFee);
```

After:

```
IERC20(WPC).forceApprove(uniswapV3SwapRouter, msg.value);  
// ... swap ...  
IERC20(WPC).forceApprove(uniswapV3SwapRouter, 0);  
if (!IPRC20(gasToken).burn(gasFee)) revert UniversalCoreErrors.PRC20operationFailed();
```

Changed `approve` to `forceApprove` (SafeERC20) for compatibility with non-standard tokens.
PRC20 `burn` return value now checked.

5. getOutboundTxGasAndFees: gasLimitUsed Return + Stale Gas Data + Zero Base Gas Limit

Before:

```
function getOutboundTxGasAndFees(address _prc20, uint256 gasLimitWithBaseLimit)  
    public view  
    returns (address gasToken, uint256 gasFee, uint256 protocolFee, uint256 gasPrice, string memory chainNamespace)  
{  
    chainNamespace = IPRC20(_prc20).SOURCE_CHAIN_NAMESPACE();  
    uint256 baseLimit = baseGasLimitByChainNamespace[chainNamespace];
```

```

// ... logic ...
gasFee = gasPrice * gasLimitWithBaseLimit;
protocolFee = protocolFeeByToken[_prc20];
}

```

After:

```

function getOutboundTxGasAndFees(address _prc20, uint256 gasLimitWithB
aseLimit)
    public view
    returns (
        address gasToken,
        uint256 gasFee,
        uint256 protocolFee,
        uint256 gasPrice,
        string memory chainNamespace,
        uint256 gasLimitUsed
    )
{
    chainNamespace = IPRC20(_prc20).SOURCE_CHAIN_NAMESPACE();
    uint256 baseLimit = baseGasLimitByChainNamespace[chainNamespace];
    if (baseLimit == 0) revert UniversalCoreErrors.ZeroBaseGasLimit();
    // ... logic ...
    _validateGasDataFreshness(chainNamespace);
    gasFee = gasPrice * gasLimitWithBaseLimit;
    protocolFee = protocolFeeByToken[_prc20];
    gasLimitUsed = gasLimitWithBaseLimit;
}

```

Three changes: (1) 6th return value `gasLimitUsed` added. (2) `ZeroBaseGasLimit` revert if base limit not configured. (3) Staleness validation via `_validateGasDataFreshness`.

6. getRescueFundsGasLimit: Stale Gas Data

Before:

```

gasFee = gasPrice * rescueGasLimit;

```

After:

```
_validateGasDataFreshness(chainNamespace);  
gasFee = gasPrice * rescueGasLimit;
```

Added staleness validation before computing gas fee.

7. setChainMeta: Zero Gas Price Allowed

Before:

```
function setChainMeta(string memory chainNamespace, uint256 price, uint256 chainHeight) external onlyUEModule {  
    gasPriceByChainNamespace[chainNamespace] = price;  
    // ...  
}
```

After:

```
function setChainMeta(string memory chainNamespace, uint256 price, uint256 chainHeight) external onlyUEModule {  
    if (price == 0) revert UniversalCoreErrors.ZeroGasPrice();  
    gasPriceByChainNamespace[chainNamespace] = price;  
    // ...  
}
```

Rejects zero gas price to prevent misconfigured chains from producing zero-fee quotes.

8. updateGasTokenPRC20 (was setGasTokenPRC20): Gas Token Update Doesn't Invalidate Gas Price

Before:

```
function setGasTokenPRC20(string memory chainNamespace, address prc20) external onlyRole(MANAGER_ROLE) {  
    if (prc20 == address(0)) revert CommonErrors.ZeroAddress();  
    gasTokenPRC20ByChainNamespace[chainNamespace] = prc20;  
    emit SetGasToken(chainNamespace, prc20);  
}
```

After:

```

function updateGasTokenPRC20(string memory chainNamespace, address prc20) external onlyRole(UVCORE_ADMIN_ROLE) {
    if (prc20 == address(0)) revert CommonErrors.ZeroAddress();
    gasTokenPRC20ByChainNamespace[chainNamespace] = prc20;
    gasPriceByChainNamespace[chainNamespace] = 0;
    emit SetGasToken(chainNamespace, prc20);
}

```

Changing the gas token now zeroes out the cached gas price, forcing a fresh `setChainMeta` call before the chain can be used again.

9. updateDefaultFeeTier (was setDefaultFeeTier): Rejects Valid 1bps Fee Tier

Before:

```

function setDefaultFeeTier(address token, uint24 feeTier) external onlyAdmin {
    if (feeTier != 500 && feeTier != 3000 && feeTier != 10000) {
        revert UniversalCoreErrors.InvalidFeeTier();
    }
    defaultFeeTier[token] = feeTier;
}

```

After:

```

function updateDefaultFeeTier(address token, uint24 feeTier) external onlyRole(UVCORE_ADMIN_ROLE) {
    if (
        feeTier != FEE_TIER_LOWEST && feeTier != FEE_TIER_LOW && feeTier != FEE_TIER_MEDIUM
        && feeTier != FEE_TIER_HIGH
    ) {
        revert UniversalCoreErrors.InvalidFeeTier();
    }
    defaultFeeTier[token] = feeTier;
    emit SetDefaultFeeTier(token, feeTier);
}

```

Now accepts fee tier `100` (1bps). Uses named constants (`FEE_TIER_LOWEST = 100` , `FEE_TIER_LOW = 500` , `FEE_TIER_MEDIUM = 3000` , `FEE_TIER_HIGH = 10000`). Emits event.

10. updateUniswapV3Addresses (was setUniswapV3Addresses): uniswapV3Quoter Stored but Never Used

Before:

```
function setUniswapV3Addresses(address factory, address swapRouter, address quoter) external onlyAdmin {
    if (factory == address(0) || swapRouter == address(0) || quoter == address(0)) {
        revert CommonErrors.ZeroAddress();
    }
    uniswapV3Factory = factory;
    uniswapV3SwapRouter = swapRouter;
    uniswapV3Quoter = quoter;
}
```

After:

```
function updateUniswapV3Addresses(address factory, address swapRouter) external onlyRole(OPERATOR_ROLE) {
    if (factory == address(0) || swapRouter == address(0)) {
        revert CommonErrors.ZeroAddress();
    }
    uniswapV3Factory = factory;
    uniswapV3SwapRouter = swapRouter;
    emit SetUniswapV3Addresses(factory, swapRouter);
}
```

`quoter` parameter removed — `uniswapV3Quoter` state variable deleted. Emits event.

11. unpause: Role Separation

Before:

```
function unpause() external onlyRole(PAUSER_ROLE) { _unpause(); }
```

After:

```
function unpause() external onlyRole(OPERATOR_ROLE) { _unpause(); }
```

Unpause restricted to `OPERATOR_ROLE` (not `PAUSER_ROLE`), preventing a compromised pauser from re-enabling the system.

12. rescueNativePC (NEW): PC Can Become Permanently Stuck

Before: Function did not exist.

After:

```
function rescueNativePC(address payable to, uint256 amount) external onlyRole(UVCORE_ADMIN_ROLE) {
    if (to == address(0)) revert CommonErrors.ZeroAddress();
    if (amount == 0) revert CommonErrors.ZeroAmount();
    if (amount > address(this).balance) revert CommonErrors.InsufficientBalance();
    (bool ok,) = to.call{value: amount}("");
    if (!ok) revert CommonErrors.TransferFailed();
    emit RescueNativePC(to, amount);
}
```

Allows admin to rescue native PC stuck in the contract (e.g., from failed swaps or refunds).

13. updateMaxStalenessByChain (NEW): Stale Gas Price Data Accepted Without Freshness Validation

Before: Function and storage did not exist.

After:

```
mapping(string => uint256) public maxStalenessByChainNamespace;

function updateMaxStalenessByChain(string memory chainNamespace, uint256 maxStaleness)
    external
    onlyRole(UVCORE_ADMIN_ROLE)
{
    maxStalenessByChainNamespace[chainNamespace] = maxStaleness;
    emit SetMaxStalenessByChain(chainNamespace, maxStaleness);
}
```

New configurable freshness window per chain. When set > 0, gas fee queries revert if chain data is older than `maxStaleness` seconds.

14. `_validateGasDataFreshness` (NEW): Stale Gas Price Data Accepted Without Freshness Validation

Before: Did not exist.

After:

```
function _validateGasDataFreshness(string memory chainNamespace) private view {
    uint256 maxAge = maxStalenessByChainNamespace[chainNamespace];
    if (maxAge == 0) return;
    uint256 observedAt = timestampObservedAtByChainNamespace[chainNamespace];
    if (block.timestamp > observedAt + maxAge) {
        revert UniversalCoreErrors.StaleGasData(observedAt, block.timestamp, maxAge);
    }
}
```

Internal staleness check called from `getOutboundTxGasAndFees` and `getRescueFundsGasLimit`.

15. `_autoSwap`: Inconsistent ERC-20 Handling + Return Values Ignored

Before:

```
IPRC20(prc20).deposit(address(this), amount);
IPRC20(prc20).approve(uniswapV3SwapRouter, amount);
// ... swap ...
IPRC20(prc20).approve(uniswapV3SwapRouter, 0);
```

After:

```
if (!IPRC20(prc20).deposit(address(this), amount)) revert UniversalCoreErrors.PRC20OperationFailed();
IERC20(prc20).forceApprove(uniswapV3SwapRouter, amount);
// ... swap ...
IERC20(prc20).forceApprove(uniswapV3SwapRouter, 0);
```

PRC20 deposit return checked. Approval now uses `SafeERC20.forceApprove` via the `IERC20` interface.

16. Removed: `setSupportedToken`, `setSlippageTolerance`, `setPauserRole`, `onlyAdmin` modifier

Removed	Reason
<code>isSupportedToken</code> mapping + <code>setSupportedToken()</code>	Unused storage — never enforced at runtime
<code>slippageTolerance</code> mapping + <code>setSlippageTolerance()</code>	Configured but never enforced in swap logic
<code>uniswapV3Quoter</code> state variable	Stored but never used
<code>setPauserRole()</code>	Use OZ <code>grantRole</code> / <code>revokeRole</code> directly
<code>onlyAdmin</code> modifier	Replaced by role-specific <code>onlyRole(...)</code>
<code>defaultDeadlineMins = 20</code> field default	Now set in <code>initialize()</code> to avoid proxy storage mismatch

17. All function renames (`set*` → `update*`)

Old Name	New Name	New Role
<code>setProtocolFeeByToken</code>	<code>updateProtocolFeeByToken</code>	<code>UVCORE_ADMIN_ROLE</code>
<code>setGasPCPool</code>	<code>updateGasPCPool</code>	<code>UVCORE_ADMIN_ROLE</code>
<code>setGasTokenPRC20</code>	<code>updateGasTokenPRC20</code>	<code>UVCORE_ADMIN_ROLE</code>
<code>setAutoSwapSupported</code>	<code>updateAutoSwapSupported</code>	<code>UVCORE_ADMIN_ROLE</code>
<code>setWPC</code>	<code>updateWPC</code>	<code>OPERATOR_ROLE</code>
<code>setUniversalGatewayPC</code>	<code>updateUniversalGatewayPC</code>	<code>OPERATOR_ROLE</code>
<code>setUniswapV3Addresses</code>	<code>updateUniswapV3Addresses</code>	<code>OPERATOR_ROLE</code>
<code>setDefaultFeeTier</code>	<code>updateDefaultFeeTier</code>	<code>UVCORE_ADMIN_ROLE</code>
<code>setDefaultDeadlineMins</code>	<code>updateDefaultDeadlineMins</code>	<code>UVCORE_ADMIN_ROLE</code>
<code>setBaseGasLimitByChain</code>	<code>updateBaseGasLimitByChain</code>	<code>UVCORE_ADMIN_ROLE</code>
<code>setRescueFundsGasLimitByChain</code>	<code>updateRescueFundsGasLimitByChain</code>	<code>UVCORE_ADMIN_ROLE</code>

Note: `setChainMeta` was NOT renamed (it's called by the UE Module, not a role-guarded setter).

18. New events added: Missing Events for Critical Admin Changes

```
event SetAutoSwapSupported(address indexed token, bool supported);
event SetWPC(address indexed oldAddr, address indexed newAddr);
```

```

event SetUniversalGatewayPC(address indexed oldAddr, address indexed newAddr);
event SetUniswapV3Addresses(address factory, address swapRouter);
event SetDefaultFeeTier(address indexed token, uint24 feeTier);
event SetMaxStalenessByChain(string chainNamespace, uint256 maxStaleness);
event RescueNativePC(address indexed to, uint256 amount);

```

Removed: `PauserRoleGranted`, `SetSupportedToken`.

CEA.sol

19. Storage layout change: Removed direct VAULT+UG addresses

Before:

```

address public pushAccount;
address public VAULT;
address public UNIVERSAL_GATEWAY;
ICEAFactory public factory;

```

After:

```

address public pushAccount;
ICEAFactory public factory;

```

`VAULT` and `UNIVERSAL_GATEWAY` are no longer stored directly. They are read live from `factory.VAULT()` and `factory.UNIVERSAL_GATEWAY()`. New view getters added:

```

function VAULT() external view returns (address) { return factory.VAULT(); }
function UNIVERSAL_GATEWAY() external view returns (address) { return factory.UNIVERSAL_GATEWAY(); }

```

20. initializeCEA: Removed direct VAULT+UG addresses

Before:

```

function initializeCEA(address _pushAccount, address _vault, address _universalGateway, address _factory) external {
    pushAccount = _pushAccount;

```

```

    VAULT = _vault;
    UNIVERSAL_GATEWAY = _universalGateway;
    factory = ICEAFactory(_factory);
}

```

After:

```

function initializeCEA(address _pushAccount, address _factory) external {
    pushAccount = _pushAccount;
    factory = ICEAFactory(_factory);
}

```

Signature reduced from 4 params to 2. VAULT and gateway read from factory at runtime.

21. onlyVault modifier: Live lookup from factory

Before:

```

modifier onlyVault() {
    if (msg.sender != VAULT) revert CEAErrors.NotVault();
    _;
}

```

After:

```

modifier onlyVault() {
    if (msg.sender != factory.VAULT()) revert CEAErrors.NotVault();
    _;
}

```

22. sendUniversalTxToUEA: Single-Call Dual Semantics + ERC-20 Approval Reset + SafeERC20 Unused

Before:

```

function sendUniversalTxToUEA(address token, uint256 amount, bytes calldata payload, address revertRecipient) external {
    if (msg.sender != address(this)) revert CommonErrors.Unauthorized();
    if (amount == 0) revert CEAErrors.InvalidInput();
}

```

```

        if (revertRecipient == address(0)) revert CEAErrors.InvalidInput
        ();
        // ...
        if (token == address(0)) {
            if (address(this).balance < amount) revert CEAErrors.Insuffici
            entBalance();
            IUniversalGateway(UNIVERSAL_GATEWAY).sendUniversalTxFromCEA{va
            lue: amount}(req);
        } else {
            if (IERC20(token).balanceOf(address(this)) < amount) revert CE
            AErrors.InsufficientBalance();
            IUniversalGateway(UNIVERSAL_GATEWAY).sendUniversalTxFromCEA(re
            q);
        }
    }
}

```

After:

```

function sendUniversalTxToUEA(address token, uint256 amount, bytes cal
ldata payload, address revertRecipient) external {
    if (msg.sender != address(this)) revert CommonErrors.Unauthorized
    ();
    if (revertRecipient == address(0)) revert CEAErrors.InvalidInput
    ();
    // ...
    address gateway = factory.UNIVERSAL_GATEWAY();

    if (amount > 0) {
        if (token == address(0)) {
            if (address(this).balance < amount) revert CEAErrors.Insuf
            ficientBalance();
            IUniversalGateway(gateway).sendUniversalTxFromCEA{value: a
            mount}(req);
        } else {
            if (IERC20(token).balanceOf(address(this)) < amount) rever
            t CEAErrors.InsufficientBalance();
            IERC20(token).approve(gateway, amount);
            IUniversalGateway(gateway).sendUniversalTxFromCEA(req);
            IERC20(token).approve(gateway, 0);
        }
    }
}

```

```

    } else {
        IUniversalGateway(gateway).sendUniversalTxFromCEA(req);
    }
}

```

Changes: (1) Removed `amount == 0` revert — zero-amount payload-only sends now allowed. (2) ERC-20 path adds `approve(amount)` before and `approve(0)` after gateway call. (3) Gateway address read from factory (not stored). (4) `SafeERC20` import removed (was unused). (5) Three-way branch: `amount>0` native, `amount>0` ERC-20, `amount==0` payload-only.

23. `_handleMulticall`: Multicall Discards Underlying Revert Reasons

Before:

```

(bool success,) = calls[i].to.call{value: calls[i].value}(calls[i].data);
if (!success) revert CEAEErrors.ExecutionFailed();

```

After:

```

(bool success, bytes memory returnData) = calls[i].to.call{value: calls[i].value}(calls[i].data);
if (!success) {
    if (returnData.length > 0) {
        assembly { revert(add(32, returnData), mload(returnData)) }
    } else {
        revert CEAEErrors.ExecutionFailed();
    }
}

```

Bubbles up underlying revert reason if available. Falls back to `ExecutionFailed()` for empty reverts.

24. `_handleSingleCall`: Single-Call Dual Semantics + Revert Reasons

Before:

```

function _handleSingleCall(...) internal {
    if (payload.length == 0) {
        emit UniversalTxExecuted(...);
        return;
    }
}

```

```

    }
    // ...
    (bool success,) = recipient.call{value: msg.value}(payload);
    if (!success) revert CEAErrors.ExecutionFailed();
}

```

After:

```

function _handleSingleCall(...) internal {
    if (payload.length == 0 && recipient == address(0)) {
        emit UniversalTxExecuted(...);
        return;
    }
    // ...
    (bool success, bytes memory returnData) = recipient.call{value: msg.value}(payload);
    if (!success) {
        if (returnData.length > 0) {
            assembly { revert(add(32, returnData), mload(returnData)) }
        } else {
            revert CEAErrors.ExecutionFailed();
        }
    }
}

```

Funds-parking now requires BOTH `payload.length == 0` AND `recipient == address(0)` (explicit signal). Revert reason bubbling added.

25. _handleMigration: CEA Migration Does Not Enforce Self-Targeting

Before:

```

function _handleMigration() internal {
    if (msg.value != 0) revert CEAErrors.InvalidInput();
    address migrationContract = factory.CEA_MIGRATION_CONTRACT();
    // ...
}

```

After:

```
function _handleMigration(address recipient) internal {
    if (recipient != address(this)) revert CEAEErrors.InvalidRecipient
    ();
    if (msg.value != 0) revert CEAEErrors.InvalidInput();
    address migrationContract = factory.CEA_MIGRATION_CONTRACT();
    // ...
}
```

Now requires `recipient == address(this)` — migration must explicitly target self.

26. executeUniversalTx param rename: txId → subTxId

Before:

```
function executeUniversalTx(bytes32 txId, bytes32 universalTxId, ...)
external payable onlyVault nonReentrant {
    if (isExecuted[txId]) revert CEAEErrors.PayloadExecuted();
    isExecuted[txId] = true;
    _handleExecution(txId, universalTxId, ...);
}
```

After:

```
function executeUniversalTx(bytes32 subTxId, bytes32 universalTxId,
...) external payable onlyVault nonReentrant {
    if (isExecuted[subTxId]) revert CEAEErrors.PayloadExecuted();
    isExecuted[subTxId] = true;
    _handleExecution(subTxId, universalTxId, ...);
}
```

Renamed for clarity — `subTxId` represents an individual sub-transaction within a universal tx.

CEAFactory.sol

27. Granular Role Separation

Refer to the full Access Control design document:  [Push Smart Contracts - AccessControl](#)

28. deployCEA: CEA init params simplified

Before:

```
ICEA(cea).initializeCEA(pushAccount, VAULT, UNIVERSAL_GATEWAY, address  
(this));
```

After:

```
ICEA(cea).initializeCEA(pushAccount, address(this));
```

Only passes `pushAccount` and factory address — CEA reads VAULT/gateway from factory at runtime.

29. Function renames and role reassignments

Old Name	New Name	Old Role	New Role
<code>setVault</code>	<code>updateVault</code>	<code>DEFAULT_ADMIN_ROLE</code>	<code>OPERATOR_ROLE</code>
<code>setUniversalGateway</code>	<code>updateUniversalGateway</code>	<code>DEFAULT_ADMIN_ROLE</code>	<code>OPERATOR_ROLE</code>
<code>setCEAMigrationContract</code>	<code>updateCEAMigrationContract</code>	<code>DEFAULT_ADMIN_ROLE</code>	<code>CEA_ADMIN_ROLE</code>
<code>setCEAProxyImplementation</code>	(unchanged name)	<code>DEFAULT_ADMIN_ROLE</code>	<code>CEA_ADMIN_ROLE</code>
<code>setCEAImplementation</code>	(unchanged name)	<code>DEFAULT_ADMIN_ROLE</code>	<code>CEA_ADMIN_ROLE</code>
<code>unpause</code>	(unchanged name)	<code>PAUSER_ROLE</code>	<code>OPERATOR_ROLE</code>

30. Removed: setPauserRole, PauserRoleGranted event

`setPauserRole` function removed. Use OZ `grantRole(PAUSER_ROLE, addr)` directly. `PauserRoleGranted` event removed from interface.

UEAFactory.sol

Test Version of UEAFactoryV0.sol - <https://github.com/pushchain/push-chain-core-contracts/blob/audit-main-fixes/src/testnetV0/UEAFactoryV0.sol>

31. Granular Role Separation

Refer to the full Access Control design document:  [Push Smart Contracts - AccessControl](#)

32. getOriginForUEA: Hardcoded Push Chain ID

Before:

```
account = UniversalAccountId({chainNamespace: "eip155", chainId: "4210
```

```
1", owner: bytes(abi.encodePacked(addr))));
```

After:

```
account = UniversalAccountId({chainNamespace: "eip155", chainId: pushChainId, owner: bytes(abi.encodePacked(addr))});
```

Configurable Push Chain ID instead of hardcoded `"42101"`. New `pushChainId` storage variable set at initialization.

33. registerUEA: Can Overwrite Existing VM Implementation

Before:

```
function registerUEA(bytes32 _chainHash, bytes32 _vmHash, address _UEA) public onlyRole(DEFAULT_ADMIN_ROLE) {
    // ... validation ...
    UEA_VM[_vmHash] = _UEA;
    emit UEARegistered(_chainHash, _UEA, _vmHash);
}
```

After:

```
function registerUEA(bytes32 _chainHash, bytes32 _vmHash, address _UEA) public onlyRole(UEA_ADMIN_ROLE) {
    _registerUEA(_chainHash, _vmHash, _UEA);
}

function _registerUEA(bytes32 _chainHash, bytes32 _vmHash, address _UEA) internal {
    // ... validation ...
    if (UEA_VM[_vmHash] != address(0)) {
        revert UEAAAlreadyRegistered();
    }
    UEA_VM[_vmHash] = _UEA;
    emit UEARegistered(_chainHash, _UEA, _vmHash);
}
```

Registration is now one-time only. Reverts with `UEAAAlreadyRegistered()` if VM hash already has an implementation.

34. updateUEAImplementation (NEW): Explicit update path for VM implementations

Before: Did not exist.

After:

```
function updateUEAImplementation(bytes32 _vmHash, address _newUEA) external onlyRole(UEA_ADMIN_ROLE) {
    if (_newUEA == address(0)) revert UEAErrors.InvalidInputArgs();
    address previous = UEA_VM[_vmHash];
    if (previous == address(0)) revert UEAErrors.InvalidInputArgs();
    UEA_VM[_vmHash] = _newUEA;
    emit UEAImplementationUpdated(_vmHash, previous, _newUEA);
}
```

Explicit update function (requires existing registration). Emits `UEAImplementationUpdated`.

35. updatePushChainId (NEW): Configurable Push Chain ID

Before: Did not exist.

After:

```
function updatePushChainId(string memory _pushChainId) external onlyRole(UEA_ADMIN_ROLE) {
    if (bytes(_pushChainId).length == 0) revert UEAErrors.InvalidInputArgs();
    pushChainId = _pushChainId;
}
```

36. Function renames and role reassignments

Old Name	New Name	Old Role	New Role
<code>setUEAProxyImplementation</code>	<code>updateUEAProxyImplementation</code>	<code>DEFAULT_ADMIN_ROLE</code>	<code>UEA_ADMIN_ROLE</code>
<code>setUEAMigrationContract</code>	<code>updateUEAMigrationContract</code>	<code>DEFAULT_ADMIN_ROLE</code>	<code>UEA_ADMIN_ROLE</code>
<code>registerNewChain</code>	(unchanged name)	<code>DEFAULT_ADMIN_ROLE</code>	<code>UEA_ADMIN_ROLE</code>
<code>registerMultipleUEA</code>	(unchanged name)	<code>DEFAULT_ADMIN_ROLE</code>	<code>UEA_ADMIN_ROLE</code>
<code>registerUEA</code>	(unchanged name)	<code>DEFAULT_ADMIN_ROLE</code>	<code>UEA_ADMIN_ROLE</code>
<code>unpause</code>	(unchanged name)	<code>PAUSER_ROLE</code>	<code>OPERATOR_ROLE</code>

37. Removed: setPauserRole, PauserRoleGranted event

Removed in favor of OZ `grantRole` / `revokeRole`.

UEA_EVM.sol

38. domainSeparator: Domain Separator Missing block.chainid

Before:

```
bytes32 public constant DOMAIN_SEPARATOR_TYPEHASH =
    0x2aef22f9d7df5f9d21c56d14029233f3fdaa91917727e1eb68e504d27072d6c
d;
// keccak256("EIP712Domain(string version,uint256 chainId,address veri
fyingContract)")

function domainSeparator() public view returns (bytes32) {
    uint256 chainId = StringUtils.stringToExactUInt256(_universalAccou
ntId.chainId);
    return keccak256(abi.encode(DOMAIN_SEPARATOR_TYPEHASH, keccak256(b
ytes(VERSION)), chainId, address(this)));
}
```

After:

```
bytes32 public constant DOMAIN_SEPARATOR_TYPEHASH =
    0xb90aaffa4b0fc25d6056f438f2c06198968eaf6723d182f5f928441117424b8
e;
// keccak256("EIP712Domain(string version,uint256 chainId,address veri
fyingContract,bytes32 salt)")

function domainSeparator() public view returns (bytes32) {
    uint256 chainId = StringUtils.stringToExactUInt256(_universalAccou
ntId.chainId);
    return keccak256(abi.encode(DOMAIN_SEPARATOR_TYPEHASH, keccak256(b
ytes(VERSION)), chainId, address(this), bytes32(block.chainid)));
}
```

Added `bytes32 salt` field containing `bytes32(block.chainid)` — prevents cross-deployment replay across Push Chain forks.

39. executeUniversalTx: payload.nonce Ignored During Signature Hashing

Before:

```
function executeUniversalTx(UniversalPayload calldata payload, bytes c
alldata signature) external nonReentrant {
    if (msg.sender != UNIVERSAL_EXECUTOR_MODULE) {
        bytes32 payloadHash = getUniversalPayloadHash(payload);
        if (!verifyUniversalPayloadSignature(payloadHash, signature))
    {
        revert UEAErrors.InvalidEVMSignature();
    }
    }
    _handleExecution(payload);
}
```

After:

```
function executeUniversalTx(UniversalPayload calldata payload, bytes c
alldata signature) external nonReentrant {
    if (payload.nonce != nonce) {
        revert UEAErrors.NonceMismatch(nonce, payload.nonce);
    }

    if (msg.sender != UNIVERSAL_EXECUTOR_MODULE) {
        bytes32 payloadHash = getUniversalPayloadHash(payload);
        if (!verifyUniversalPayloadSignature(payloadHash, signature))
    {
        revert UEAErrors.InvalidEVMSignature();
    }
    }
    _handleExecution(payload);
}
```

Explicit nonce check before signature verification. Applies to all callers (including module).
Provides descriptive `NonceMismatch(expected, provided)` error.

UEA_SVM.sol

40. domainSeparator: SVM Domain Separator Encodes chainId Incorrectly + Missing block.chainid

Before:

```
bytes32 public constant DOMAIN_SEPARATOR_TYPEHASH_SVM =
    0x3aefc31558906b9b2c54de94f82a9b2455c24b4ba2b642ebb545ea2cc64a1e4
b;
// keccak256("EIP712Domain_SVM(string version,string chainId,address v
erifyingContract)")

function domainSeparator() public view returns (bytes32) {
    return keccak256(abi.encode(
        DOMAIN_SEPARATOR_TYPEHASH_SVM,
        keccak256(bytes(VERSION)),
        _universalAccountId.chainId, // raw string, not hashed
        address(this)
    ));
}
```

After:

```
bytes32 public constant DOMAIN_SEPARATOR_TYPEHASH_SVM =
    0x038a4fd0ee5950f0ea6d28f116a885fc5e376a8d1a939f7a9bea48f4f13fabb
1;
// keccak256("EIP712Domain_SVM(string version,string chainId,address v
erifyingContract,bytes32 salt)")

function domainSeparator() public view returns (bytes32) {
    return keccak256(abi.encode(
        DOMAIN_SEPARATOR_TYPEHASH_SVM,
        keccak256(bytes(VERSION)),
        keccak256(bytes(_universalAccountId.chainId)), // now hashed
        address(this),
        bytes32(block.chainid)
    ));
}
```

Two fixes: (1) `chainId` now hashed with `keccak256` per EIP-712 spec for dynamic types. (2) Added `bytes32 salt` field with `block.chainid`.

41. executeUniversalTx: payload.nonce Ignored During Signature Hashing

Identical to UEA_EVM change:

After:

```
function executeUniversalTx(UniversalPayload calldata payload, bytes c
alldata signature) external nonReentrant {
    if (payload.nonce != nonce) {
        revert UEAErrors.NonceMismatch(nonce, payload.nonce);
    }
    // ... signature verification ...
    _handleExecution(payload);
}
```

UEAProxy.sol

42. initializeUEA: Proxy Initialization Validation Inconsistent

Before:

```
function initializeUEA(address _logic) external initializer {
    address currentImpl = getImplementation();
    if (currentImpl != address(0)) {
        revert UEAErrors.InvalidCall();
    }
    // ...
}
```

After:

```
function initializeUEA(address _logic) external initializer {
    if (_logic == address(0)) {
        revert UEAErrors.InvalidCall();
    }

    address currentImpl = getImplementation();
    if (currentImpl != address(0)) {
        revert UEAErrors.InvalidCall();
    }
    // ...
}
```

Added zero-address check for `_logic` parameter before proceeding.

PRC20.sol

Note: NEW implementation of TESTNET PC20 is - <https://github.com/pushchain/push-chain-core-contracts/blob/audit-main-fixes/src/testnetV0/PRC20V0.sol>

43. transferFrom: Balance Transfer Before Allowance Validation

Before:

```
function transferFrom(address sender, address recipient, uint256 amount) external returns (bool) {
    _transfer(sender, recipient, amount);

    uint256 currentAllowance = _allowances[sender][msg.sender];
    if (currentAllowance < amount) revert PRC20Errors.LowAllowance();
    unchecked { _allowances[sender][msg.sender] = currentAllowance - amount; }
    emit Approval(sender, msg.sender, _allowances[sender][msg.sender]);
    return true;
}
```

After:

```
function transferFrom(address sender, address recipient, uint256 amount) external returns (bool) {
    uint256 currentAllowance = _allowances[sender][msg.sender];
    if (currentAllowance < amount) revert PRC20Errors.LowAllowance();
    unchecked { _allowances[sender][msg.sender] = currentAllowance - amount; }
    emit Approval(sender, msg.sender, _allowances[sender][msg.sender]);

    _transfer(sender, recipient, amount);
    return true;
}
```

Allowance check now happens BEFORE the transfer (checks-effects-interactions pattern).

44. deposit: PRC20 Direct Mint Bypasses UniversalCore Pause

Before:

```
function deposit(address to, uint256 amount) external returns (bool) {
    if (msg.sender != UNIVERSAL_CORE && msg.sender != UNIVERSAL_EXECUTOR_MODULE) {
        revert PRC20Errors.InvalidSender();
    }
    _mint(to, amount);
    emit Deposit(abi.encodePacked(UNIVERSAL_EXECUTOR_MODULE), to, amount);
    return true;
}
```

After:

```
function deposit(address to, uint256 amount) external returns (bool) {
    if (msg.sender != UNIVERSAL_CORE && msg.sender != UNIVERSAL_EXECUTOR_MODULE) {
        revert PRC20Errors.InvalidSender();
    }
    if (PausableUpgradeable(UNIVERSAL_CORE).paused()) {
        revert PRC20Errors.CorePaused();
    }
    _mint(to, amount);
    emit Deposit(abi.encodePacked(msg.sender), to, amount);
    return true;
}
```

Two changes: (1) Checks `UniversalCore.paused()` before minting — prevents mints when protocol is paused. (2) Emit uses actual `msg.sender` instead of hardcoded

`UNIVERSAL_EXECUTOR_MODULE`.

45. setName / setSymbol: Missing Events

Before:

```
function setName(string memory newName) external onlyUniversalExecutor
{ _name = newName; }
```

```
function setSymbol(string memory newSymbol) external onlyUniversalExecutor { _symbol = newSymbol; }
```

After:

```
function setName(string memory newName) external onlyUniversalExecutor {
    string memory oldName = _name;
    _name = newName;
    emit NameUpdated(oldName, newName);
}
function setSymbol(string memory newSymbol) external onlyUniversalExecutor {
    string memory oldSymbol = _symbol;
    _symbol = newSymbol;
    emit SymbolUpdated(oldSymbol, newSymbol);
}
```

Added `NameUpdated` and `SymbolUpdated` events.

WPC.sol

46. totalSupply: totalSupply Inflated by Force-Sent Native PC

Before:

```
function totalSupply() public view returns (uint256) {
    return address(this).balance;
}
```

After:

```
uint256 private _totalSupply;

function totalSupply() public view returns (uint256) {
    return _totalSupply;
}
```

Tracks supply via internal counter instead of `address(this).balance`. Prevents inflation via `selfdestruct` or coinbase force-sends.

47. deposit / withdraw: Supply tracking + withdraw Uses .transfer (2300 Gas)

Before:

```
function deposit() public payable {
    balanceOf[msg.sender] += msg.value;
    emit Deposit(msg.sender, msg.value);
}

function withdraw(uint256 wad) public {
    require(balanceOf[msg.sender] >= wad, "");
    balanceOf[msg.sender] -= wad;
    payable(msg.sender).transfer(wad);
    emit Withdrawal(msg.sender, wad);
}
```

After:

```
function deposit() public payable {
    _totalSupply += msg.value;
    balanceOf[msg.sender] += msg.value;
    emit Deposit(msg.sender, msg.value);
}

function withdraw(uint256 wad) public {
    if (balanceOf[msg.sender] < wad) revert WPCErrors.InsufficientBalance();
    balanceOf[msg.sender] -= wad;
    _totalSupply -= wad;
    (bool ok,) = msg.sender.call{value: wad}("");
    if (!ok) revert WPCErrors.TransferFailed();
    emit Withdrawal(msg.sender, wad);
}
```

Changes: (1) `_totalSupply` tracking in deposit/withdraw. (2) `payable.transfer(wad)` replaced with low-level `.call{value}` (no 2300 gas limit). (3) `require` replaced with custom errors.

48. transferFrom: Empty Revert Messages

Before:

```
function transferFrom(address src, address dst, uint256 wad) public re
turns (bool) {
    require(balanceOf[src] >= wad, "");
    if (src != msg.sender && allowance[src][msg.sender] != type(uint25
6).max) {
        require(allowance[src][msg.sender] >= wad, "");
        // ...
    }
}
```

After:

```
function transferFrom(address src, address dst, uint256 wad) public re
turns (bool) {
    if (balanceOf[src] < wad) revert WPCErrors.InsufficientBalance();
    if (src != msg.sender && allowance[src][msg.sender] != type(uint25
6).max) {
        if (allowance[src][msg.sender] < wad) revert WPCErrors.Insuffi
cientAllowance();
        // ...
    }
}
```

Empty string `require` replaced with custom errors for better failure observability.

Errors.sol (libraries)

49. New errors added

```
// PRC20Errors
error CorePaused();

// UniversalCoreErrors (added)
error ZeroBaseGasLimit();
error StaleGasData(uint256 observedAt, uint256 nowTimestamp, uint256 m
axAge);
error PRC20operationFailed();

// UniversalCoreErrors (removed)
```

```

error InvalidSlippageTolerance(); // removed

// WPCErrors (NEW library)
library WPCErrors {
    error InsufficientBalance();
    error InsufficientAllowance();
    error TransferFailed();
}

// StringUtilsErrors (NEW library)
library StringUtilsErrors {
    error EmptyString();
    error NonDigitCharacter();
}

// UEAErrors (added)
error NonceMismatch(uint256 expected, uint256 provided);
error UEAAlreadyRegistered();

```

Utils.sol (StringUtils)

50. stringToExactUInt256: Custom Errors for Gas Efficiency

Before:

```

require(len > 0, "Empty string cannot be converted.");
// ...
require(c >= 48 && c <= 57, "Non-digit character found.");

```

After:

```

if (len == 0) revert StringUtilsErrors.EmptyString();
// ...
if (c < 48 || c > 57) revert StringUtilsErrors.NonDigitCharacter();

```

String `require` messages replaced with custom errors.

TESTNET: UniversalCoreV0.sol (testnetV0 — storage-compatible upgrade)

51. Deprecated state variables

```
// Was:
mapping(address => uint256) public slippageTolerance;
address public uniswapV3Quoter;
mapping(address => bool) public isSupportedToken;

// Now:
mapping(address => uint256) private __deprecated_slippageTolerance;
address private __deprecated_uniswapV3Quoter;
mapping(address => bool) private __deprecated_isSupportedToken;
```

Slots retained for storage layout compatibility. Marked `private` with `__deprecated_` prefix.

52. setChainMeta: signature change (removed `observedAt` param)

Before:

```
function setChainMeta(string memory chainNamespace, uint256 price, uint256 chainHeight, uint256 observedAt) external onlyUEModule {
    timestampObservedAtByChainNamespace[chainNamespace] = observedAt;
}
```

After:

```
function setChainMeta(string memory chainNamespace, uint256 price, uint256 chainHeight) external onlyUEModule {
    timestampObservedAtByChainNamespace[chainNamespace] = block.timestamp;
}
```

`observedAt` parameter removed. Now uses `block.timestamp` directly — prevents stale/manipulated external timestamps.

53. All function renames (set* → update*)

Same rename pattern as UniversalCore v1. Functions renamed: `setProtocolFeeByToken`, `setGasPCPool`, `setGasTokenPRC20`, `setAutoSwapSupported`, `setWPC`, `setUniversalGatewayPC`, `setUniswapV3Addresses`, `setDefaultFeeTier`, `setDefaultDeadlineMins`, `setBaseGasLimitByChain`, `setRescueFundsGasLimitByChain`.

54. Removed: setSupportedToken, setSlippageTolerance

Unused functions removed.

55. Fee tier constants added

```
uint24 public constant FEE_TIER_LOWEST = 100;
uint24 public constant FEE_TIER_LOW = 500;
uint24 public constant FEE_TIER_MEDIUM = 3000;
uint24 public constant FEE_TIER_HIGH = 10000;
uint256 public constant MAX_SLIPPAGE_BPS = 5000;
```

TESTNET: UEAFactoryV0.sol (testnetV0)

56. getOriginForUEA: configurable pushChainId

Same change as UEAFactory v1 — hardcoded `"42101"` replaced with `pushChainId` storage variable.

57. New: updatePushChainId, pushChainId state variable

```
string public pushChainId;

function updatePushChainId(string memory _pushChainId) external onlyOwner {
    if (bytes(_pushChainId).length == 0) revert UEAEErrors.InvalidInputArgs();
    pushChainId = _pushChainId;
}
```

58. Function renames (set* → update*)

```
setUEAProxyImplementation → updateUEAProxyImplementation, setUEAMigrationContract →
updateUEAMigrationContract.
```

59. PauserRoleGranted event moved to contract body

Event removed from `IUEAFactory` interface (v1 no longer emits it). Added locally in `UEAFactoryV0` contract to maintain ABI compatibility for testnet.

Review Tasks

Contract	Function / Area	What to check	Owner
UniversalCore	getOutboundTxGasAndFees	Confirm SDK usage + impact of new return value(s)	Zaryab Afser
UniversalCore	_validateGasDataFreshness	Ensure max staleness + observedAt data are set so outbound/rescue quoting works	Aman Gupta / Nilesh Gupta
UniversalCore	setChainMeta	Confirm whether zero gas price / new checks affect current flows	Aman Gupta / Nilesh Gupta
UniversalCore	updateMaxStalenessByChain	Set up chain infra/job to update the staleness window value	Aman Gupta / Nilesh Gupta
CEA	Storage layout change (VAULT/UG removed)	Decide if fresh CEA deployment is required on all chains; validate current testnet coverage + feasibility	Aman Gupta / Nilesh Gupta / Zaryab Afser
CEA	sendUniversalTxToUEA	Update SDK tests for new semantics (zero-amount payload-only + approvals)	Shoaib Mohammed
CEA	_handleMulticall	Update SDK tests for bubbled revert reasons / error handling	Shoaib Mohammed
CEA	_handleSingleCall	Update SDK tests for new funds-parking condition + revert bubbling	Shoaib Mohammed
CEA	_handleMigration	Update SDK tests for new recipient self-targeting requirement	Shoaib Mohammed
UEAFactory	getOriginForUEA	Check if SDK calls this; confirm breaking change risk with configurable pushChainId	Shoaib Mohammed
UEA_EVM	domainSeparator	Confirm breaking SDK change + module-side verification for salt/block.chainid update	Shoaib Mohammed / Nilesh Gupta
UEA_SVM	domainSeparator	Confirm breaking SDK change + module-side verification for chainId hashing + salt	Shoaib Mohammed / Nilesh Gupta
UniversalCoreV0 (testnet)	Deprecated state variables	Sanity-check deprecated slots + storage-compatibility approach	Nilesh Gupta
PC20V0	New Testnet version of PC20 tokens	Note: NEW implementation of TESTNET PC20 is - https://github.com/pushchain/push-chain-core-contracts/blob/audit-	@Nilesh Gupta

Contract	Function / Area	What to check	Owner
		<u>main-</u> <u>fixes/src/testnetV0/PRC20V0.sol</u>	
UEAFactory	Test version of UEAFactory		@Nilesh Gupta
UVCore	New Testnet Version		@Nilesh Gupta